PolyRMC

An auditable pipeline for ACD light-scattering measurement, and the agent that drives it.

This describes what the system is, what each part is allowed to do, and -- more usefully -- what each part refuses to do and why. The refusals are the design. A measurement pipeline that always produces a number is not a feature; most of the engineering here is in producing nothing, loudly, when the honest answer is that a number cannot be justified.

1. The one rule

Models judge; they never compute. Every reported quantity is produced by deterministic code that a person can read. A model may choose which window to smooth over, which runs to compare, which figure answers a question -- decisions where judgement is genuinely required and where being wrong is visible. No model multiplies anything.

The consequence worth stating: the entire measurement path runs with no model and no API key. `intake`, `process`, `plot` and `--judge static` are complete without one. The agent is an interface to the pipeline, never a dependency of it.

2. Three tiers

Tier	What it is	What it may not do
0	Deterministic computation: ingest, time reconstruction, splice, censoring, smoothing, optics, M0, virial fit.	Import anything above it. No model can reach it.
1	Judgement over a bounded choice, made by a model or by the deterministic `StaticJudge`.	Compute a reported value. It picks among options tier 0 produced.
2	Orchestration: the session, the tool surface, the conversation.	Commit anything without a person approving it.

Tier 0 has no dependency on the tiers above it, and a test enforces that (`tests/test_tier0_purity.py`). It is the part that must still be correct in ten years when the model vendors have all changed.

3. Getting data in

`polyrmc.tabular.open_table` recognises four kinds of file, tried in order, because the specific formats declare themselves and the general one cannot:

Kind	What	Notes
`kaleidagraph`	KaleidaGraph `.QDA`	Binary, reverse-engineered. Appendix A.
`processed`	A boundary CSV this pipeline wrote	Declares its own header length; legacy column names renamed on read.
`argen`	An ARGEN acquisition	All 16 scattering channels kept, not just ls8.
`delimited`	Anything else with a header row	Delimiter sniffed, and reported as sniffed.

An unrecognised file must contain at least one column with real numbers before it is called a table. Without that rule the sniffer accepts prose: a sentence splits on whitespace into as many columns as it has words, and a ragged second line leaves an all-empty column that pandas types as float, so a dtype check calls two lines of English numeric.

4. Declarations

A run is configured by statements a person makes about the experiment, never by inference from the trace. The canonical case is `dn_dc`: 0.185 for a protein, 0.150 for a polysaccharide. Guessing it processes the run under a physical model nobody chose, and the resulting number looks entirely reasonable.

Where a value is genuinely disputed, it is recorded as a question with candidates and sources rather than resolved quietly. The same principle governs inference elsewhere: the run registry will report a material inferred from `dn_dc`, but labels it `inferred from dn_dc` and never as a declaration. See Appendix C for what that currently surfaces in this lab's archive.

5. The pipeline

`run_part1` ingests and cleans: header length read from the file rather than assumed, time reconstructed and checked for monotonicity, detector channels selected, splices and censoring recorded, anchors proposed. It stops before choosing a smoothing window, which is a judgement.

`run_part2` performs the virial fit over an emitted CSV. Figures are drawn alongside both, each in its own `try`, so one broken plot costs that plot and names it rather than taking the set down.

6. Provenance

Every processed run writes a `.provenance.json` sidecar beside its CSV: the configuration, the ingest decisions, every anomaly and excision, the baseline candidates whether or not any were used, the M0 window, and the branch taken at each unresolved design question. Schema version 2 adds declarations and the path register; an absent `paths` key means *not recorded*, which is a different fact from *no branches taken*.

`polyrmc.rehydrate` reads a finished run back from its CSV and sidecar, well enough to redraw it, without reprocessing. Reprocessing under today's code is not the same act as reading what was recorded then.

7. The agent

The route is the model's to choose. The loop passes the conversation to the model, runs the tools it asks for, and hands back results. The guarantees never lived in the route, so giving up the route costs nothing: validation stays in the tool surface, which rejects in code, and tier 0 stays unreachable.

Tier	Examples	Approval
read	`get_state`, `search_past_runs`, `open_data_file`, `describe_plot_file`, `read_notes`	None. Approving each read trains a researcher to approve without looking.
compute	`load_run`, `sweep_windows`, `plot_figure`, `plot_columns`, `extract_parameters`	None. Produces results, decides nothing.
commit	`set_declaration`, `pick_window`, `apply_baseline_correction`, `emit_csv`	**Every one stops for a person.**

A rejection is a result, not a crash. `Rejected` carries what failed, by how much, and what would be valid, and goes back to the model as the outcome of its call so the next attempt can be correct. The loop is capped, and the cap is reported rather than hidden.

8. Providers

The package ships no key and brokers no model access. One `ProviderSpec` table covers Gemini, Claude and OpenAI; with no `--provider` the session uses whichever is keyed. `polyrmc setup` takes a key and spends one small call to confirm it opens a door, because the alternative is a researcher discovering a bad paste three questions into a session, where the failure looks like the agent.

There is deliberately **no failover** onto a second key. That key is also the researcher's, and moving their session onto it spends money they never agreed to spend.

Failure is typed, because the fix differs and the loop must tell a brain that will not answer apart from a tool that refuses. A refusing tool is fed back for the model to correct; a dead provider leaves nothing to correct with, so it ends the session, says what was spent, and stops. Nothing committed is lost.

Category	The fix
Out of quota or rate-limited	A bill, or a wait. Served as 403 by one vendor and 429 by another, so it is matched on meaning.
Rejected key	A paste. `polyrmc setup`.
Stale model id	A `--model`, with the override named.
Unreachable	A wait.

Keys stay out of sight throughout: read with `getpass` and never echoed, storing to `.env` asked separately from accepting the key because a shared or teaching machine is a normal place to run this, `/providers` reports presence and never value, and anything shaped like a credential is redacted out of an SDK error before it reaches a terminal somebody may paste into a lab channel.

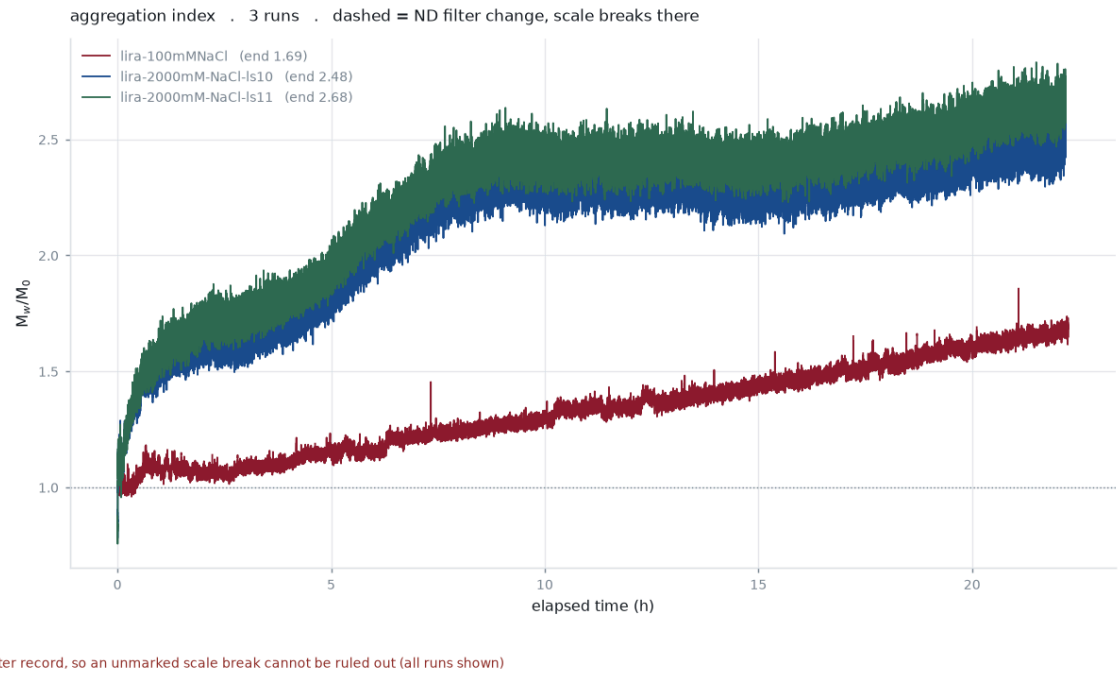
9. Figures

Two kinds, and the difference is stamped into the image because a PNG outlives the conversation that made it.

Reviewed figures (`plot_figure`) draw one specific thing and know what it means, which is what lets them label their own axes and flag their own caveats. Gaps in the record are printed onto the figure itself -- and filtered to what that figure actually reads, since a caveat about something the drawing never consulted is noise, and small red text that is usually noise stops being read at all.

Exploratory plots (`plot_columns`) draw whatever columns they are told to, from a file nothing has processed. They carry a corner stamp saying so.

The comparison figure plots M_w/M_0 and never raw counts. Two runs' counts differ by concentration, gain, and ND-filter position; drawing them together puts two numbers side by side that were never measured in the same units, which is the most convincing wrong figure this package could produce.



Three runs on one axis. The two 2000 mM channels track each other; 100 mM sits well below. The ND-filter caveat rides inside the image.

10. What tier 0 actually does

Sixteen modules under `polyrmc/tier0/`, none of which imports anything above it:

Module	Concern
`argen_io`	Reading an acquisition: header length taken from the file, delimiter detected, time reconstructed from a wrapping clock and checked for monotonicity.
`columns`, `classify`	Which columns are scattering channels, and what kind of experiment this is.
`detect`, `censoring`, `splice`	Finding anomalies, removing what cannot be measured, and recording the gap so it is never drawn or averaged across.
`smoothing`	The safe band of window lengths, and the residual diagnostics that say whether a window tracked the trace.
`baseline`, `anchors`, `landmarks`	Drift models, the anchor candidates offered for them, and the structural points of a run.
`optics`, `m0`, `dilution`	The optical constant, the normalising M_0, and concentration.
`fit_range`, `virial`, `consistency`	Choosing the fitting range, the virial expansion, and the checks that catch a fit that cannot be true.

Four ways to get a plausible wrong number

These are the traps this tier exists to close, and each one produces output that looks entirely reasonable:

- **Absolute thresholds.** A spike detector tuned to counts on one instrument at one gain silently stops detecting when either changes. Thresholds are expressed relative to the signal's own scale.
- **Global noise estimates.** A single noise figure for a run whose noise grows with signal will over-flag the quiet start and under-flag the loud end. Noise is estimated locally.
- **Correlated residuals.** A smoothing window can fit beautifully by every pointwise measure while systematically lagging the trace. The residual autocorrelation is what catches it, and it is checked rather than assumed.
- **Unconditional baseline correction.** Subtracting a drift model that was never justified moves every downstream number. Correction is a commit-tier decision, is recorded with the candidate set that was on offer, and is reversible.

The anomaly taxonomy

Five kinds, and the distinction matters because the right response differs. A detector reports what it found; what to do about it is a commit-tier decision.

Kind	What it means
`transient_spike`	A brief excursion that recovers to baseline — a bubble, a stirrer artifact. Removable.
`baseline_drift`	A slow shift with no recovery. Removable only with a justified drift model.
`saturation_artifact`	The detector reached full scale. These samples are **limits, not measurements**, and averaging them in reports a value the instrument could not see.
`structural_noise`	Noise whose character changes, which usually means the sample changed rather than the instrument.
`splice_gap`	Data removed. A multi-sample gap breaks the plotted line rather than being bridged, so what was removed is never drawn as though it were signal.

11. The tool surface in full

Thirty-three tools. The tier decides whether a person is asked, and nothing else about a tool changes with it.

read (16) — no approval

`get_state`, `get_history`, `get_shortlist`, `get_intake_form`, `list_anomalies`, `list_conflicts`, `list_blocked`, `list_anchor_candidates`, `search_past_runs`, `get_past_run`, `compare_to_past_runs`, `search_figures`, `open_data_file`, `describe_plot_file`, `read_notes`, `write_note`

compute (8) — no approval, decides nothing

`load_run`, `sweep_windows`, `check_declarations`, `fit_drift`, `extract_parameters`, `verify_output`, `plot_figure`, `plot_columns`

commit (9) — every one stops for a person

`set_declaration`, `resolve_conflict`, `set_config`, `pick_window`, `set_anomaly_decision`, `select_anchors`, `apply_baseline_correction`, `revert_baseline_correction`, `emit_csv`

`sweep_windows` computes the safe band and **does not choose one**; `fit_drift` reports what applying a drift model would do and **subtracts nothing**. The split between computing an option and taking it is the whole shape of this surface.

12. A run, end to end

1. `polyrmc intake` — the experiment is declared. Nothing is inferred from the trace.
2. `load_run` — ingest and clean. Stops before the smoothing window.
3. `check_declarations` — every declaration tested against the trace; conflicts recorded whether or not they are resolved.
4. `sweep_windows`, then `pick_window` — the band is computed, a person or judge picks from it.
5. Anomalies and anchors decided, baseline applied only if justified.
6. `emit_csv` — the boundary artifact, plus a provenance sidecar and the per-run figure set.
7. `extract_parameters` — the virial fit over that CSV.
8. `verify_output` — the numbered checks, reported as passed, skipped or failed.

An unresolved declaration conflict blocks emission. That is the point at which the system prefers producing nothing.

13. Testing

The suite runs with **no API key, no network, and no instrument file**. Synthetic data is generated with known answers — a virial series with a known M_w and A_2, an ARGEN file with a wrapping clock — so a test can assert the right number rather than the current one.

The agent loop is exercised end to end against a scripted provider, including the paths that matter most: a tool that rejects, a commit the researcher declines, a handler that raises, and the step cap holding. Binary format readers are tested against files the suite writes itself from the documented layout, so the writer and the reader must agree about a format neither of them owns.

Appendix A — the .QDA format

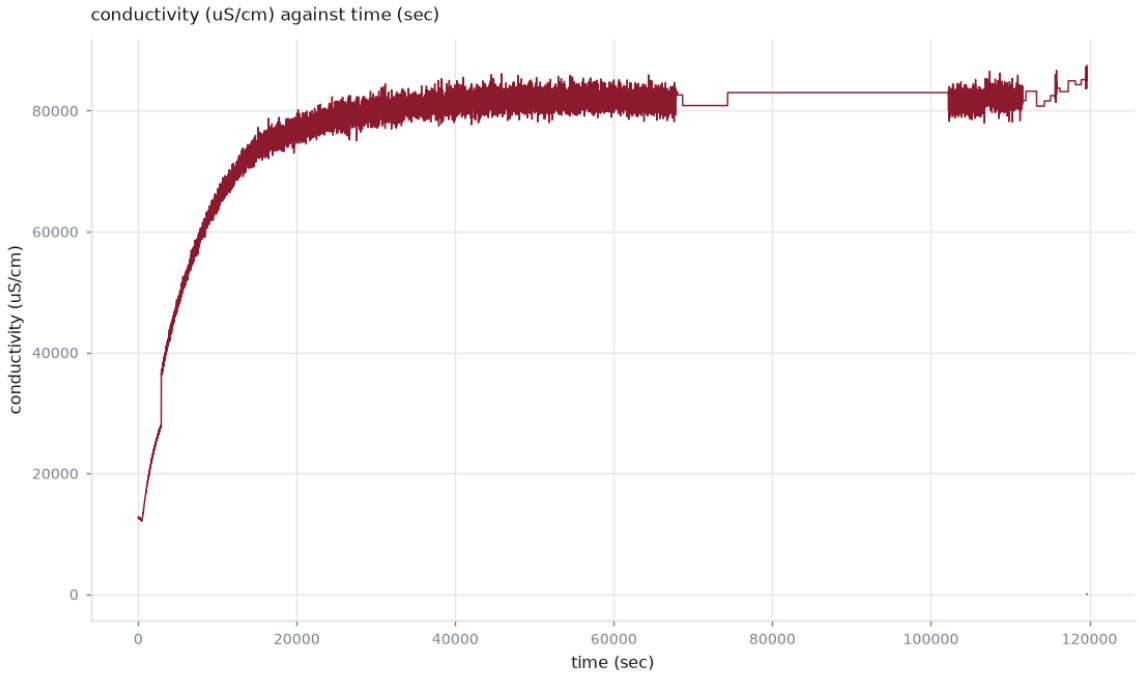
Reverse-engineered from this lab's files; there is no published specification. Big-endian throughout, which is the giveaway that KaleidaGraph began on 68k Macs. Values are IEEE 754 **single** precision, so a column read back carries about seven significant digits regardless of what is printed after it.

```
0x000  u16   unused (000c observed)
0x002  u16   column count
0x200  ...   column table, 6 bytes per column, opening with a u32 row count
       ...   name table, 40 bytes per column, at 0x200 + columns * 6
       ...   data begins at 0x200 + columns * 46

per column, in order:
   data    n * 4 bytes, big-endian float32
   flags   n * 2 bytes, one word per value: 1 filled, 0 blank
   record  136 bytes: signature 0e 02 01 00 05 00 00 01, then the name
```

Three traps, each of which produced a plausible wrong table before it was caught:

- **Nothing sits at a fixed offset.** Both tables scale with the column count, so a 21-column file puts its names 66 bytes further along. Reading the 10-column offset lands mid-table, where bytes decode as tidy small floats.
- **The record's name is authoritative, not the table position.** A column type this reader does not handle leaves no record, and everything after it shifts by one -- labelling a temperature column "stirring speed", which plots perfectly well.
- **These files contain stored plots.** A reader that assumed every gap between records was column data read 1.3 MB of plot layout as a smooth column of zeros. The flag mask is what distinguishes a column from a picture.



exploratory plot -- columns chosen by hand, nothing processed or reviewed . conductivity.QDA (kaleidagraph)

A conductivity curve read directly out of conductivity.QDA.

Appendix B — the .QPC format, and its limit

A `.QPC` is a KaleidaGraph *graph*. **It contains no data.** The shipped examples are 16–28 KB against multi-megabyte data files, carry no column records at all, and name their source by filename. What can be recovered is what was plotted and how; the numbers are in the `.QDA`.

Recoverable: axis titles and series names, the curve fit with the initial guesses it was given (for example `m1+(m2-m1)/(1+(x/m3)^m4);m1=75;m2=5;m3=30;m4=4`), the analytic partial derivatives stored beside it, and

the source table with a timestamp.

Text is stored two ways in the same file, which is why one scan cannot find all of it. Labels are **Pascal strings** -- a length byte, then that many characters, not NUL-terminated -- so a short value written over a longer one leaves the old tail in place, and a naive reader reports `Displacement (mm)ress (MPa)`. Fit equations are plain C strings, so the length-byte scan finds only shifted fragments of them.

The two formats are told apart by **column records, not the header**: 13 of the 15 example plots open with `000f` but two older ones open with `000c`, which is also what every data file opens with. Every `.QDA` has column records; no `.QPC` has any.

There is no `.QDR` format. KaleidaGraph writes `.QDA` (data), `.QPC` (graph), `.QPL` (layout), `.QSC` (script), `.QST` (style) and `.QCT` (colour table) -- taken from the application's own file dialogs. A request for `.QDR` is answered by name, pointing at the two that exist.

Appendix C — known gaps

- **Schema drift.** 65 of 120 sidecars in this lab's archive declare `experiment_type: "fixed_concentration"`, which the current schema does not define. They stay readable and overlayable; the configuration is reported as unreadable rather than translated to the nearest current spelling, because which experiment a past run was is a statement about the experiment.
- **39 runs carry the polysaccharide `dn_dc` of 0.150 while being named for liraglutide, a peptide.** One of those two facts is wrong. The system reports the inference and its basis; it does not resolve this.
- **The `elapsedtime` column is not read** from ARGEN exports in KaleidaGraph form: it is text stored at ten bytes per row and leaves no column record. It is reported by name, never silently skipped.
- **The tier-1 `--judge model` path is Gemini-only.** It predates the provider table and has not been moved onto it. `--judge static` is the offline path either way, and is what every test runs.
- **The series-comparison figure is not exposed.** Producing it needs summary statistics (`plateau_swing`, `at_full_scale`) that no production code currently derives from a sidecar. Defining them is a judgement about the experiment and is left to be asked rather than invented.

Appendix D — module map

```
src/polyrmc/
  tier0/        deterministic computation; imports nothing above it
  tier1/        judged decisions (ModelJudge, StaticJudge)
  tier2/        orchestration entry points
  agent/
    orchestrator.py   the loop: model chooses, tools run, results return
    toolspec.py       the 33 tools, their tiers and schemas
    toolcall.py       provider table; typed provider failures
    budget.py         requests and tokens counted against a ceiling
    lanes.py          the LLM and VLM judging lanes
    render.py         how state is shown to the model
  memory/
    registry.py       index over past runs' provenance sidecars
    notes.py          what the agent concluded, kept between sessions
  intake.py       declarations, the intake form, conflicts
  pipeline.py     run_part1 (clean) and run_part2 (extract)
  tabular.py      opening any data file as a table
  kaleidagraph.py .QDA and .QPC readers
  rehydrate.py    reading a finished run back off disk
  io_csv.py       the boundary CSV, written and read
```

```
provenance.py   the sidecar
figures.py      the per-run set and the cross-run comparison
plotting.py     the two-panel overview
verify.py       the numbered output checks
paths.py        where this installation keeps its files
```

Appendix E — glossary

ACD	Automatic continuous dilution: concentration is varied during a single acquisition rather than across separate samples.
ARGEN	The multi-sample light-scattering instrument. Writes a delimited text file with a declared header length and sixteen scattering channels; this lab analyses `ls8`.
Boundary CSV	The processed output: six fixed columns with the source header carried on top. The stable artifact everything downstream reads.
dn/dc	Refractive-index increment. 0.185 for a protein, 0.150 for a polysaccharide. Declared, never inferred.
M_0	The normalising value at the start of a run. It divides every value in the aggregation index, so which window produced it must be reconstructible.
M_w/M_0	The aggregation index: each run divided by its own M_0, which is what makes two runs comparable when their raw counts are not.
ND filter	A neutral-density filter. When it moves, the channel is rescaled by an unrecorded factor and the trace either side is not on one scale.
Sidecar	The `.provenance.json` written beside each processed CSV, recording every decision taken to produce it.
A_2	The second virial coefficient, from the concentration dependence of scattering.